

Determining optimal performance-to-power efficiency given temperature constraints in high-performance GPUs

Group Members:

Samih Dalvi	[dalvis3]	[400387262]
Alexander Diab-Liu	[diabliua]	[400370788]
Dylan Pergentile	[pergentd]	[400369172]

Submitted on: April 6th, 2026

CHEMENG 4H03– Big Data Methods and Modelling in Chemical and Materials Engineering

Instructor: Dr. Kamil A. Khan

Abstract

One of the biggest constraints affecting Graphical Processing Units performance in cryptocurrency mining and other high-functioning practices is thermal management. Continuous operation at large volumes increases the risk of hardware degradation and thermal throttling. In this investigation, a publicly available GPU performance metrics dataset of nearly 25, 000 recorded instances of Bitcoin mining workloads are put through a comprehensive thermal management framework. The program includes standard scaling, Principal component analysis through the Singular Value Decomposition, different regression models, and Particle Swarm Optimization to determine maximum GPU efficiency while maintaining 5°C thermal safety margin below throttling thresholds.

After an explained variance analysis, a total of four principal components were found to be the main contributors to temperature outputs. Principal component, Ordinary Least Squares, ridge linear, and ridge polynomial regressions were all applied to the dataset to determine the strongest predictive performance of GPU temperature. The polynomial ridge model was found to be the best with a mean absolute error of 1.175 °C and a R^2 value of 0.718. The trained polynomial predictor was then embedded with a particle Swarm Optimization fitness function which found that the optimal configuration achieved was a clock speed of 3.97MHz and a power consumption of 2.83W, resulting in a maximized efficiency of 1.4028 MHz/W. The results of this investigation prove that machine learning and optimization techniques can work together to extend GPU lifetime and reduce cooling costs.

Table of Contents

Abstract.....	2
1. Introduction.....	4
1.1 Goals.....	5
1.2 Problem Formation.....	5
2. Methods.....	7
2.1 Data Preprocessing.....	7
2.2 Standard Scaling.....	8
2.3 Dimensionality Reduction Via Principal Component Analysis.....	8
2.4 Regression and Predictive Modelling.....	9
2.5 Optimization.....	10
3.1 Libraries.....	11
3.2 Data Preprocessing.....	11
3.2 Dimensionality Reduction Via PCA.....	12
3.3 Regression.....	12
3.4 PSO.....	13
4. Results and Discussion.....	14
4.1 Data Characteristics.....	14
4.2 PCA.....	15
4.3 Regression Models.....	16
4.4 Optimization.....	20
References.....	23
Appendix A: Additional Figures.....	25
Appendix B: Python Code.....	28

1. Introduction

Graphical Processing Units (GPUs) have become a foundational computing technology with many applications and uses in both professional and personal settings. One such area being data mining. GPUs have successfully improved the scalability of data mining algorithms to address increasingly larger dataset sizes in numerous applications, including Bitcoin mining [1]. Due to the strenuous and consistent nature of data mining, a common limitation is the degradation of the GPU over time [2]. Even when properly cooled, GPUs lifespan is limited to only a few years while under consistent usage, which is why thermal management is critical to the profitability of GPU usage.

Looking to the future, GPUs will continue to be an important tool for new technological advancements including Artificial Intelligence (AI) [3], machine learning, scientific simulation and high-performance computing [3]. If GPU usage will continue to increase in years to come, it is valuable to learn more about the underlying metrics that affect the thermal capabilities of GPUs, and how to best manage them to preserve the lifetime of the units.

To further investigate, a data set of GPU performance metrics was obtained for different scenarios including Bitcoin mining, gaming, password cracking, and normal usage. Within each dataset are metrics including temperature, memory, power, fan speed, core clock and other measured values related to GPU performance. Focusing on the Bitcoin mining data set, reducing the data to its principal components should help determine which metrics are the key contributors to GPU temperature. From there, predictive models could be developed to determine when certain parameters can give rise to higher temperatures.

Using this information, the ability to optimize GPU metrics for the best performance-to-power ratio would be valuable information for developing and designing a thermal management system for GPUs. It is hopeful that the results of the predictive and optimization efforts will be beneficial to future GPU usage to prevent GPU throttling points in high performance computing data centres, while also improving efficiencies and reducing costs for cooling and replacements.

1. 1 Goals

The primary goals of the investigation into GPU performance metrics include:

- Use the Principal Component Analysis (PCA) to lower the dimensionality of the data set and determine the most heavily weighted metrics contributing to GPU performance
- Use regression models to predict temperature rise given certain parameters
- Compare the effectiveness of different regression models employed to determine the best predictor of temperature rise
- Implementation of optimization technique to determine the optimal performance (frequency) to power ratios that maximize computation with 5-degree Celsius safety margin for temperature

1.2 Problem Formation

Let the GPU performance dataset be represented by the matrix:

$$X \in \mathbb{R}^{n \times p}$$

Where n is the number of observations in the dataset and p is the number of recorded GPU performance metrics. Within X , every row represents a recorded observation of GPU operation, and every column represents a specific variable.

The variable of interest is temperature, which can be represented as:

$$T \in \mathbb{R}^n$$

With T_i being the measured temperature at the i^{th} observation.

The GPU performance metrics can exhibit correlations between each other which can result in multicollinearity. To mitigate this problem, PCA can be used to break down the original X matrix into a set of matrices that still account for most of the variance in the data. The X matrix can be represented as:

$$X = U\Sigma V^T$$

Using the principal components a regression can be completed to develop a predictive model. However, to determine the accuracy of this regression model, it will be compared to another

regression that uses training and testing sets of the raw data to determine the best predictor. The predictive model's goal is to develop a function of f where:

$$y(x) = f(x) \approx T$$

The final goal is to solve a constrained optimization problem for performance-to-power as follows:

$$Max(\eta(x)) = \frac{f}{P}$$

Subject to Thermal Safety ($\hat{y}(x) \leq T_{limit} - T_{margin}$) and Hardware Constraints

Table 1: Notation Table for symbols defined in the scope of this project:

Symbol	Definition
$\mathbf{x}$	Input vector (clock, power, etc.)
$\mathbf{y}$	Measured GPU temperature in Celsius
$\hat{\mathbf{y}}$	Predicted GPU temperature from ML model
$\mathbf{f}$	GPU Clock Frequency (MHz)
$\mathbf{P}$	Power Consumption (Watts)
μ	Penalty term
η	Efficiency term (Performance-per-watt)
$\mathbf{T}_{limit}$	Hardware Thermal Throttling Threshold
$\mathbf{T}_{margin}$	Hardware Thermal Throttling Safety Margin (Celsius)

2. Methods

To fully investigate the factors contributing to GPU temperature during high performance workloads, a detailed approach was developed. This approach can be broken down into five steps including data pre-processing, dimensionality reduction of the dataset via PCA, design and implementation of regression techniques for predictive modeling, evaluation of predictive models to determine the best predictor, and implementation of an optimization technique to determine most effective GPU parameters.

2.1 Data Preprocessing

The dataset used for this investigation, titled “GPU Performance Metrics for Usage Scenarios” is a publicly available dataset from Kaggle.com[4]. The dataset contains GPU metrics measured across four usage scenarios, including Bitcoin mining, gaming, password cracking, and normal GPU usage. The Bitcoin mining subset was selected as the primary dataset for this investigation because it is relevant to high-sustained-load thermal management, which aligns with the project goals. The dataset contains 24,957 observations across 12 columns.

Table 2: List of GPU performance metrics and their status

Feature	Status
Memory used	Retained
Memory total	Dropped: Constant value for all observations
Power consumption (watts)	Retained
GPU utilities	Retained
Temperature	Retained
Fan speed	Retained
Core clock	Retained
Memory clock	Retained
Pcie_rx_MB	Retained
Pcie_tx_MB	Retained
Latency_ms	Dropped: Constant value for all observations
Timestamp	Dropped: Does not provide any valuable information in terms of thermal GPU state

A total of three features were dropped as they provided no additional or relevant information (or zero variance) in terms of the thermal state of the GPU. Additionally, any rows containing missing values were dropped entirely from the dataset. In all, nine features were retained, which included the response variable temperature. A correlation matrix was also created to justify and validate the need for a PCA based on strong pairwise correlations among features.

2.2 Standard Scaling

Before performing PCA, the feature matrix (excluding the temperature column) was standardized using the zero-mean (see equation below):

$$x_i = \frac{x_i - \mu}{\sigma}$$

Where μ is the column mean, σ is the standard deviation, both of which were computed over the training dataset. The scaling ensures that the features measured in different units/scales contribute equally, preventing higher-magnitude features from dominating the PCA decomposition.

2.3 Dimensionality Reduction Via Principal Component Analysis

The performance metrics data for GPUs are often highly correlated, which is to say that changes in some variables can produce changes in others. For example, increased power consumption and clock speed may correspond to a higher temperature, or an increased fan speed may correspond to lower temperatures. PCA reduces the number of dimensions in large datasets to principal components that retain most of the original information [5]. So, instead of investigating many different variables, the first few components containing the majority of the data's variation are investigated [6]. Through the visualization of these principal components, similarities between samples can be explored, or in this case, the important variables that are the largest contributors to variance are determined [6].

After scaling, the PCA is performed via the computation of the Singular Value Decomposition (SVD). The SVD involves breaking down the original matrix into three simpler matrices to reveal its underlying structure [5].

The SVD is mathematically expressed as:

$$X = U\Sigma V^T$$

- U is an orthogonal matrix, as its columns are perpendicular and have a unit length. U is made up of the left singular values, and each column of U indicates how strongly each data point contributes to the principal component direction [5].
- Σ is a diagonal matrix with singular values. The values are arranged in descending order and larger the singular value is, the more variance in the data is explained by its corresponding pattern [5].
- V is an orthogonal matrix with perpendicular and linearly independent columns. V^T defines the direction in which the variance occurs [5].

Following the computation of the SVD, the explained variance of each principal component can be calculated via:

$$variance_i = \frac{\sigma_i^2}{n - 1}$$

$$explained\ variance = \frac{variance_i}{total\ variance}$$

- σ_i is the singular value
- n is the number of rows in the x matrix

From here, a cumulative variance plot was created to determine the number of principal components (k=4), which cumulatively account for 90% of the explained variance. This was then used to create the T-score matrix, which houses the principal components [7].

2.4 Regression and Predictive Modelling

After reducing the dimensions of the dataset into its principal components, a few different linear regression techniques were utilized to create temperature predictive models, and their performances were compared using Mean Absolute Error (MAE) and the R^2 value. The four methods used were PCR + OLS, OLS (using raw features), Ridge regression (linear features), and Ridge regression using polynomial features. All four models were created using an 80/20 train/test split. For the ridge regression methods, hyperparameter tuning using grid search was performed over $\mu \in \{0.001, 0.05, 0.1, 1, 5, 10, 100, 1000\}$, evaluating MAE on the held-out test set for each candidate. The μ minimizing test MAE was retained for the final model.

2.5 Optimization

Particle Swarm Optimization (PSO) was selected as the optimization algorithm for this problem for several reasons. The feasible region defined by the hardware operating bounds is a continuous, bounded, non-convex search space in eight dimensions. The thermal constraint introduces a highly nonlinear boundary (the level set of the polynomial ridge predictor at $T_{\text{limit}} - T_{\text{margin}}$) that cannot be handled analytically. PSO is a derivative-free metaheuristic that requires only fitness function evaluations and is well-suited to such constrained, nonlinear spaces [12].

PSO was modeled after the collective intelligence of social animals, such as a flock of birds dispersing and collaborating to locate resources such as food or shelter. Each individual is responsible for discovering their own territory and communicating important findings to other members of the flock. The movement decisions of each animal is appropriately determined by their own first-hand discoveries as well as the information it receives from other members of the flock.

Similarly, each particle in PSO will traverse the space where each dimension represents a tracked feature of the input - in this case different GPU operating parameters such as clock speed, power, or memory used – as it searches for an optimum of a desired output feature, in this case η representing performance-to-power. Each particle as it searches for the optimum, will keep track of its own personal best value, as well as the global best value found amongst the entire swarm. These two values will govern the decision-making process of where the particle moves in the future, as shown by the following mathematical equations [12].

$$v_{j,i+1} = w * v_{j,i} + c_1 * r_1 * (p_{bestj} - x_{j,i}) + c_2 * r_2 * (g_{best} - x_{j,i})$$

This shows the value of velocity of particle j at iterations $i+1$ where:

- $w = 1.1$ is the inertia weight, the measure of how difficult it is for the particle to change direction [8]
- $c_1 = 1.5$ is the cognitive component, the weight that pulls the particle towards its own personal best [9]
- $c_2 = 1.5$ is the social coefficient, the weight pulling the particle towards the global best [9]
- $r_1, r_2 \sim \text{Uniform}(0, 1)$ are independent random variables sampled at each update

$$x_{j,i+1} = x_{j,i} + v_{j,i+1}$$

This equation simply updates the next position given the particle's current position and velocity.

The effectiveness of the PSO in locating an optimal hardware state depends entirely on the definition of the fitness function, which serves as the numerical representation of the "resource" the swarm is attempting to locate. In this implementation, the fitness function is designed to maximize the performance-per-watt of the GPU while strictly adhering to a thermal safety envelope.

$$\eta = \frac{f}{p}$$

3.0 Implementation

3.1 Libraries

Building from the methodology of the investigation, the follow steps for the data pre-processing, PCA, regression, and optimization were numerically implemented in Python 3 using a Jupyter Notebook. The external libraries used include:

- **NumPy 2.0.2.:** For all matrix operations, including SVD, regression coefficient calculations, and PSO
- **Pandas 2.2.3:** For data loading, preprocessing, and statistics
- **Matplotlib 3.10.0 / Seaborn 0.13.2:** For the correlation heatmap, PCA scree and scatter plots, regression actual-vs-predicted plots, residual plots, and the PSO convergence curve
- **scikit-learn 1.6.1 (sklearn):** PolynomialFeatures for degree-2 feature expansion.

It should be noted that all regression and PCA computations were implemented from scratch using NumPy; sklearn was not used for any ML model fitting.

3.2 Data Preprocessing

For data pre-processing, all empty rows were dropped along with three unused columns: *timestamp*, *mem_total*, and *latency_ms*. Following this, the correlation coefficient between variables was calculated, and a heatmap shown in *Appendix A* was generated to observe their relationships.

Since the temperature is the response variable being measured, the temperature column was separated from the matrix as a response vector y . Next, standard scaling was manually implemented to standardize all variables before further analysis.

3.2 Dimensionality Reduction Via PCA

The economy SVD was computed according to the methodology described in *Section 2.3* after which the cumulative sum was plotted against the number of components. A cumulative explained variance threshold of 90% was chosen to as a value within the range of typical thresholds [10]. The number of components satisfying the 90% variance threshold is four based on the plot in *Figure 1*.

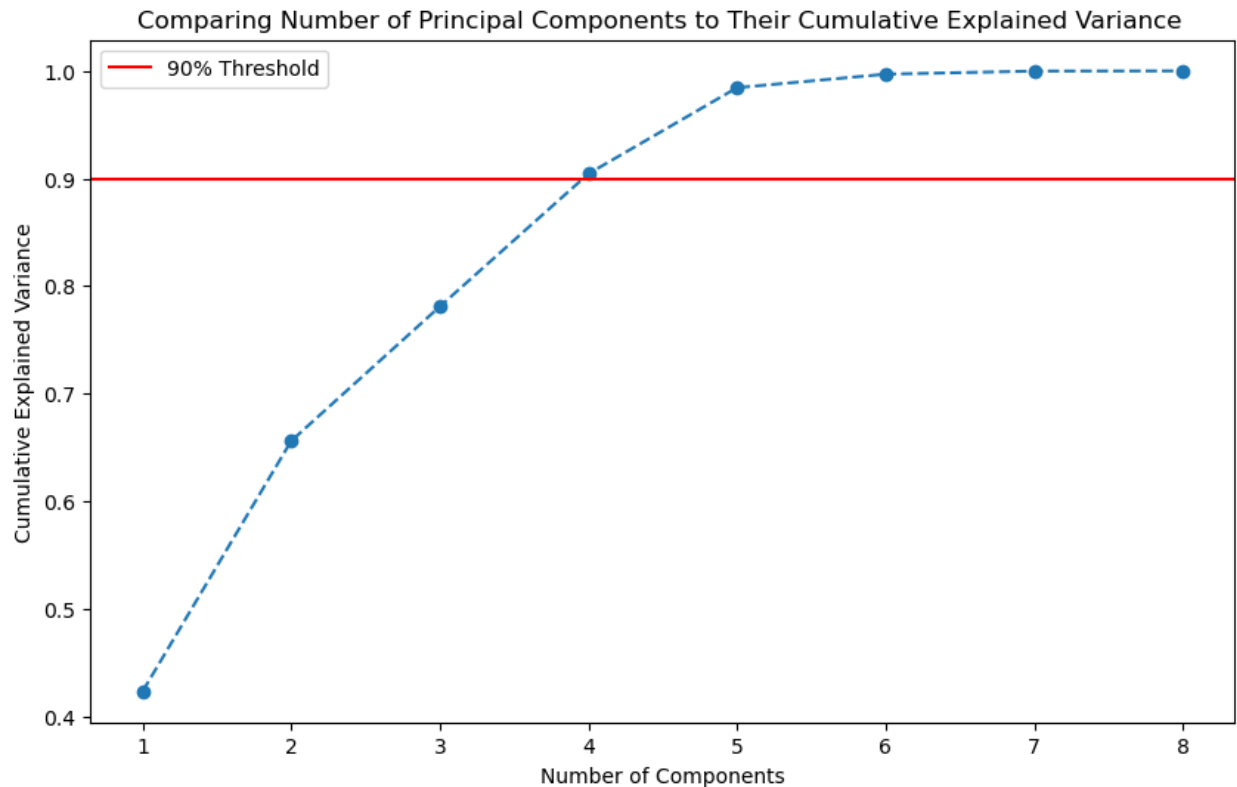


Figure 1: Comparison plot of the number of principal components needed to meet the specified cumulative explained variance threshold.

3.3 Regression

The training and test data split was implemented with a random permutation of the rows to generate an 80% training set and 20% test set. The OLS function solves the equation below.

$$\theta^T \theta a = \theta^T y, \quad (a^* = [\theta^T \theta]^{-1} \theta^T y)$$

whereas the ridge regression function adds μI to the left-hand column in an attempt to reduce multicollinearity.

$$a^* = [\theta^T \theta + \mu I]^{-1} \theta^T y \text{ for some } \mu > 0$$

The polynomial ridge regression used second degree polynomial terms from the original inputs to include the original variables as well as their squares and interaction terms [11]. To prevent data leakage, which is when the model can learn information from the test set while training, the polynomial transform was fit to only the training data and then using the same transformation applied to the test data. A grid search was then conducted to find the best ridge regularization parameter μ . From this search, a total of eight potential μ values were considered, and after computing the Mean Absolute Error (MAE) on the test set for each, a μ of 1.0 was chosen as the best option.

3.4 PSO

The Particle Swarm Optimization algorithm was implemented from scratch using NumPy. The initialization involved 20 particles and 30 iterations. The bounds were defined using the minimum and maximum values of the data to ensure particles remained within physical hardware limits. Initial positions were sampled using a uniform distribution across the bounds, and initial velocities were zero.

For each iteration, the follow steps were performed:

1. Feature Expansion: Each particle's position was transformed into the second-degree polynomial feature space using the pre-fitted PolynomialFeatures object to remain consistent with the regression model
2. Temperature Prediction: The temperature was predicted using the optimized weights from the Polynomial Ridge Regression model
3. Fitness Evaluation: If the predicted temperature was below the 78°C safety threshold, the fitness was evaluated as f/P . A small epsilon ($1e-7$) was added to prevent division by zero. If the threshold was violated, a penalty of $-1e9$ was applied.
4. Updates: Personal and global bests were updated based on the highest fitness scores.

5. Velocity and Position Clippings: After updating velocities and positions, np.clip was used to ensure particles did not drift outside the valid operating range

4. Results and Discussion

4.1 Data Characteristics

Table 3 summarizes the statistical properties of the raw Bitcoin mining GPU dataset after preprocessing. Looking specifically at the variable of interest, temperature, we can see that it exhibits relatively low variance. With a mean of 56.43 °C and a standard deviation of 3.04 °C, it suggests that the GPU remains at some steady state (within a small range) during Bitcoin mining. The low variance also implies that even small errors in temperature prediction can result in problems. For example, even a prediction that is off by 2-3 °C can lead to safety margin violations.

Table 3: List of GPU performance metrics and their status

Feature	Count	Mean	Std	Min	25%	Median	Max
mem_used (GB)	24,957	3.96	0.19	0.16	3.97	3.97	4.00
power_watts (W)	24,957	30.49	8.58	2.83	26.94	27.12	65.90
gpu_util (%)	24,957	99.01	9.70	0.00	100.0	100.0	100.0
temperature (°C)	24,957	56.43	3.04	50.00	54.00	56.00	68.00
fan_speed (RPM)	24,957	32765	207	0	32767	32767	32767
core_clock (MHz)	24,957	1773	132	300	1785	1785	1785

The correlation heatmap in *figure 2* shows strong positive correlation between power_watts and temperature. This theoretically makes sense as power dissipation is usually the main driver for temperature increases in GPU [13]. We also observe a moderate to strong negative correlation between pcie_rx_mb and temperature. This suggests that maybe data transfer-heavy workloads are less compute-intensive (less heat generated by GPU). All the other features don't really show any strong correlations with temperature, suggesting that they carry limited predictive signal for the thermal model but were retained for completeness.

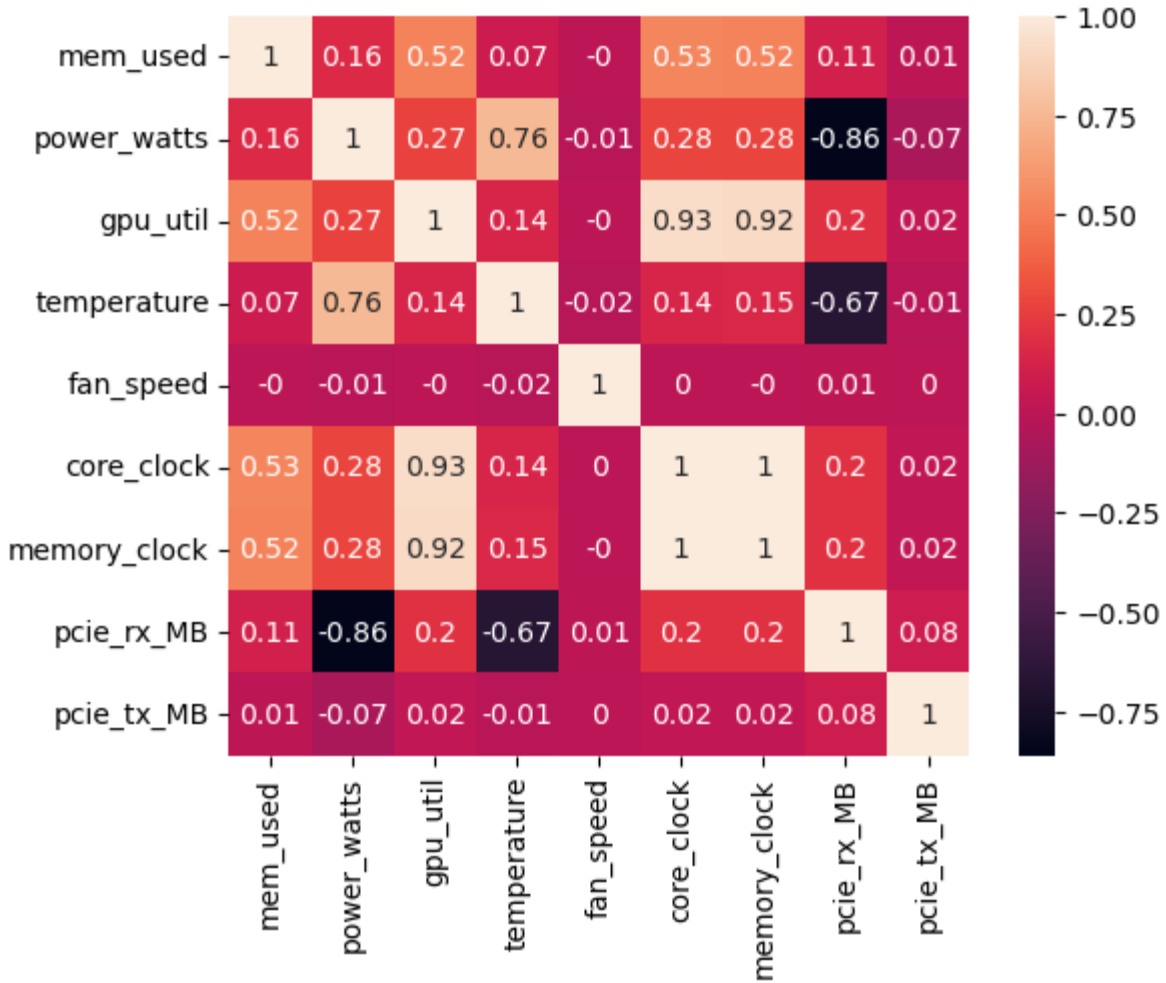


Figure 2: Correlation heatmap depicting the correlation among the nine features

4.2 PCA

As can be seen in *figure 3*, where the first two principal components are plotted along with the respective temperature corresponding to that specific data point, we see that the observations are densely clustered around $PC1 \approx 0$. This indicates that most of the GPU operating states during Bitcoin mining are highly similar (possibly caused by constant high GPU usage during the process). Additionally, we can also notice clear trend in the temperature gradient. We notice that observations in the upper-left cluster are coloured in yellow/orange (corresponding to temperatures greater than 62°C), and cooler temperatures are depicted by blue and purple near the bottom right end of the plot (less than 55°C). This general trend helps confirm that $PC2$ carries some meaningful thermal information. The fact that temperature variation is not clearly

separated along PC1 alone, despite PC1 explaining $\sim 43\%$ of the total variance, supports the decision to retain four principal components rather than just 1 or 2. Although the plot does not provide conclusive information about the effect of the other features on temperature, it can be concluded that a regression model operating only on the first two principal components would discard predictive information.

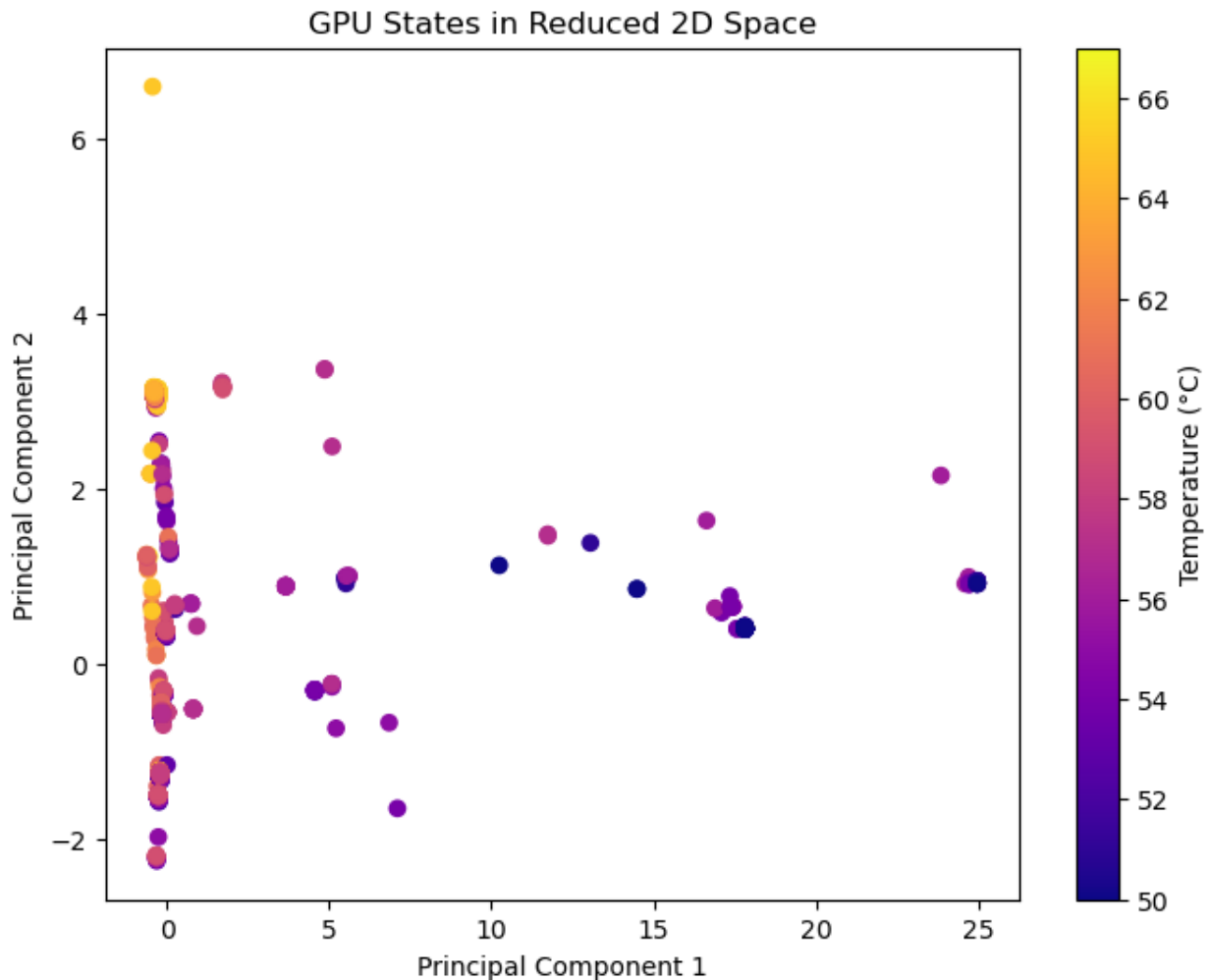


Figure 3: Plot depicting the first two principal components, with each observation coloured by its recorded GPU temperature

4.3 Regression Models

To evaluate the best regression model, all four of the regression models of PCR (OLS on PC scores), OLS, ridge, and ridge plus polynomial features are evaluated on the held-out test set based on their MAE seen in *Figures 4 -7*. The Residual plots for each of these regressions can be found in *Appendix A*.

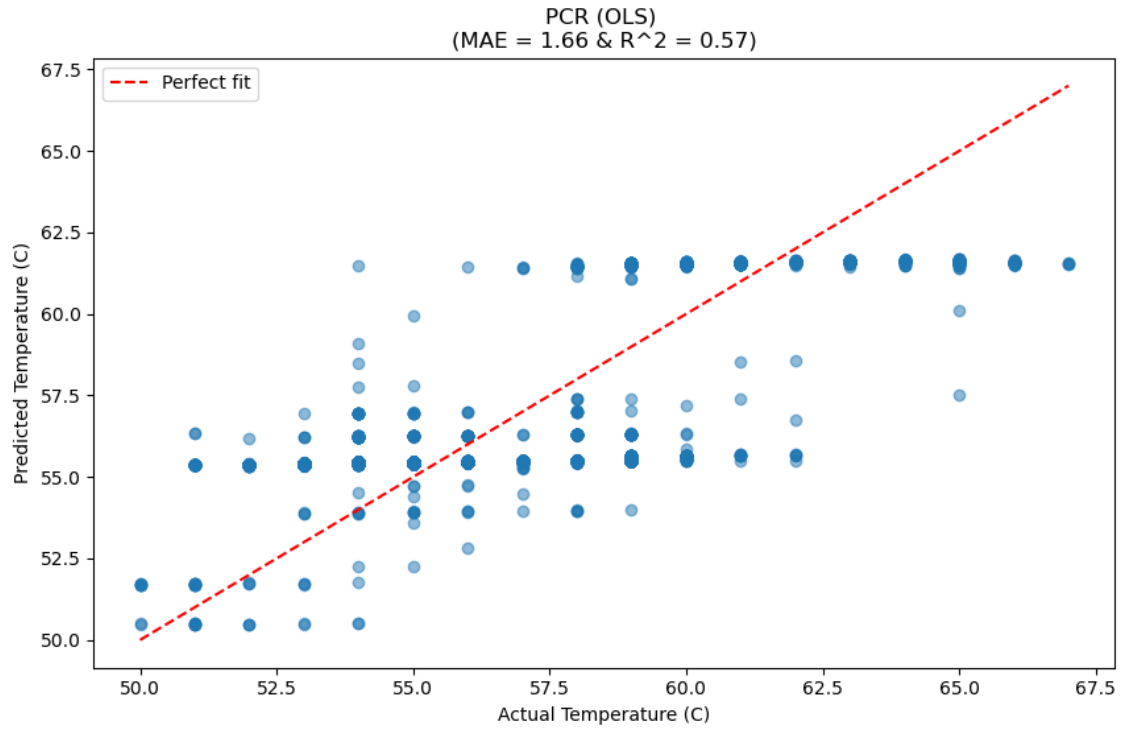


Figure 4: PCR on the test data set with an MAE of 1.66 and R^2 value of 0.57

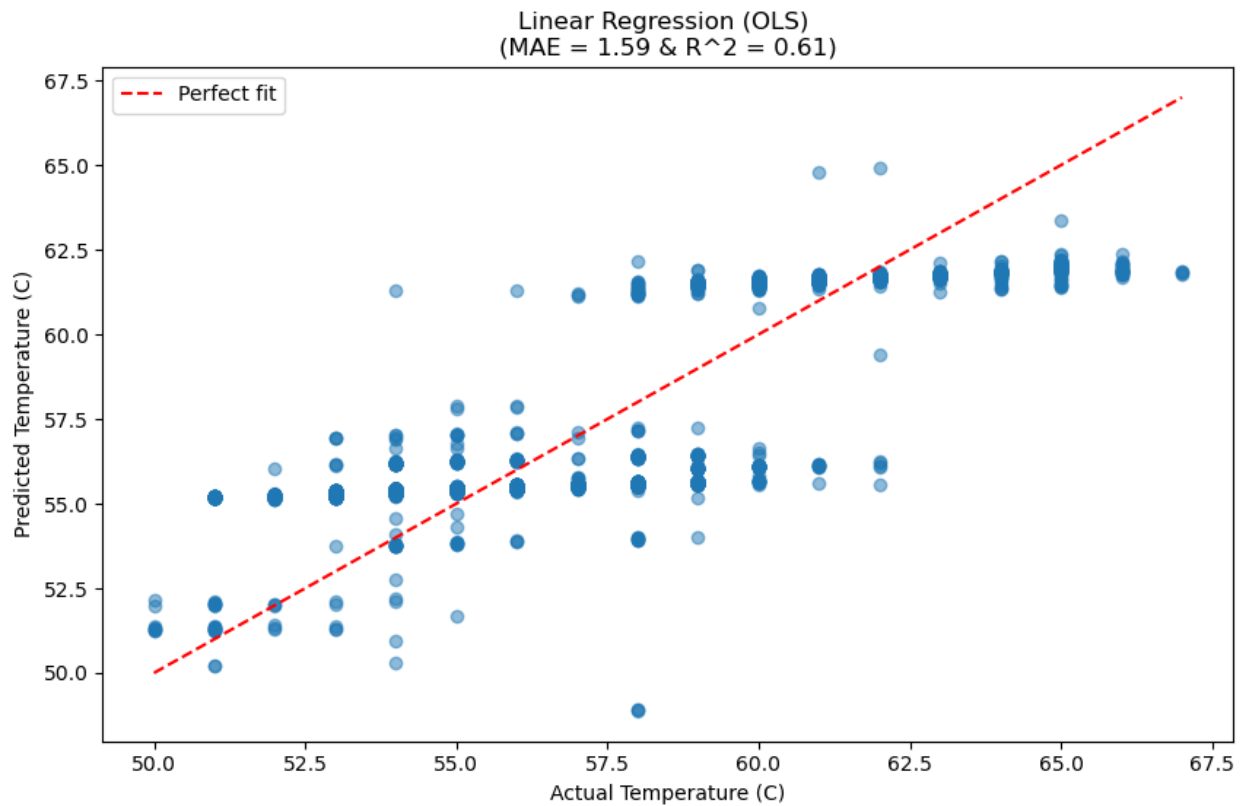


Figure 5: OLS from raw data on the test data set with an MAE of 1.59 and R^2 value of 0.61

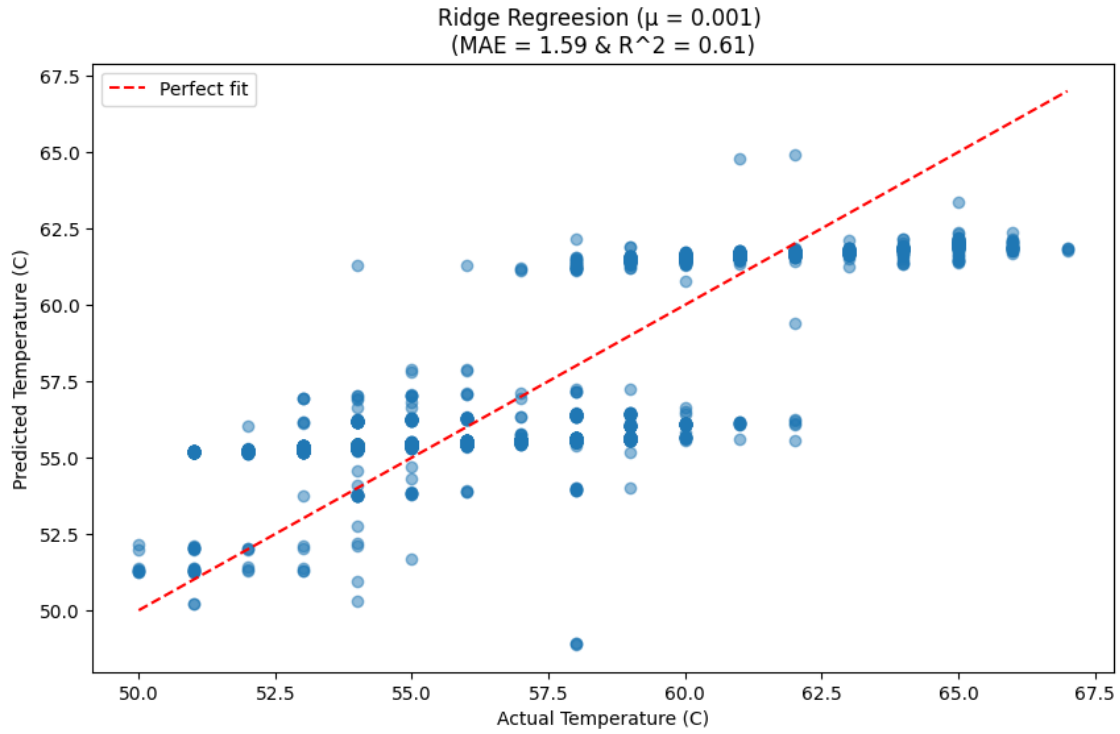


Figure 6: Ridge regression from raw data on the test data set with an MAE of 1.59 and R^2 value of 0.61

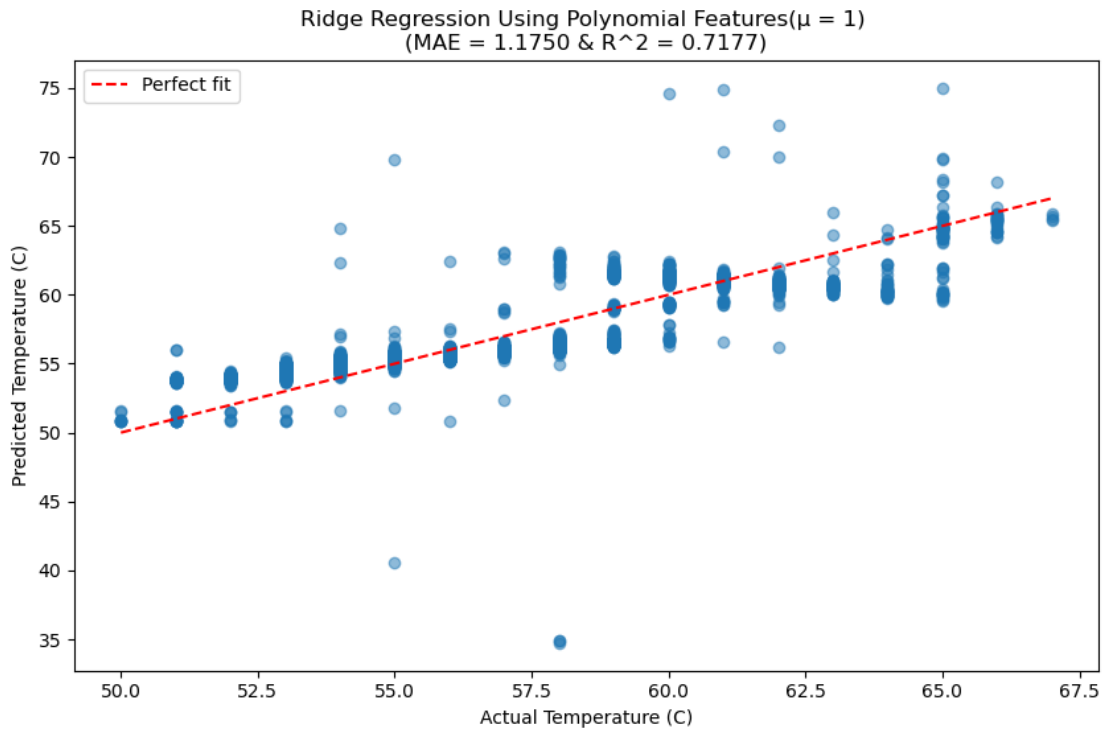


Figure 7: Ridge Regression from raw data on the test data set with an MAE of 1.1750 and R^2 value of 0.7177

The results of the four different regression models have been summarized in *Table 3* below.

Table 4: Summary of regression model and their performance results

Model	MAE (°C)	R ²	μ
PCR (OLS on PC scores)	~1.67	~0.54	N/A
OLS (raw features)	~1.59	~0.607	N/A
Ridge (raw features)	~1.586	~0.607	0.001
Ridge + Poly Features (deg=2)	~1.175	~0.718	1.0

PCR was the weakest regression of the four, with an MAE of roughly 0.08 °C higher than OLS on raw features. While multicollinearity was a concern in the raw data, this regression likely still performed the weakest due to data lost from only including 4 principal components. While 91% of the variance was captured by these 4 components, the remaining 9% could possibly have a disproportional number of signals that contribute to temperature prediction. Essentially, the movement into the principal component space may have discarded data of low variance that still carried information valuable to predicting temperature.

The OLS on raw data and the ridge regression resulted in nearly identical performances with identical R² values and a MAE of 1.59 °C and 1.586 °C for OLS and ridge, respectively. The μ value of 0.01 for the ridge regression confirmed that the 19,965 training samples were well representative of the overall data set and very minimal regularization was required.

The best performing regression was the polynomial ridge model. Its MAE of 1.175 °C and R² of 0.781 is markedly better than the OLS and ridge models with a 25.9% decrease in MAE between them. The expansion into a degree-2 polynomial has allowed the model to include interaction terms, most notably the power watts and fan speed terms which are used to determine the operating temperatures. The μ value of 1 indicates that the polynomial model required regularization when expanded to prevent over-fitting in the dataset.

Looking at the residuals, the ridge polynomial model reduces heteroscedasticity versus the three other linear models, meaning this model was more consistent in its prediction errors. However, some variability still exists around the middle temperatures of 50-60 °C. This could be accounted for via factors that were not accounted for in the model, like ambient temperature or changes in cooling effectiveness.

4.4 Optimization

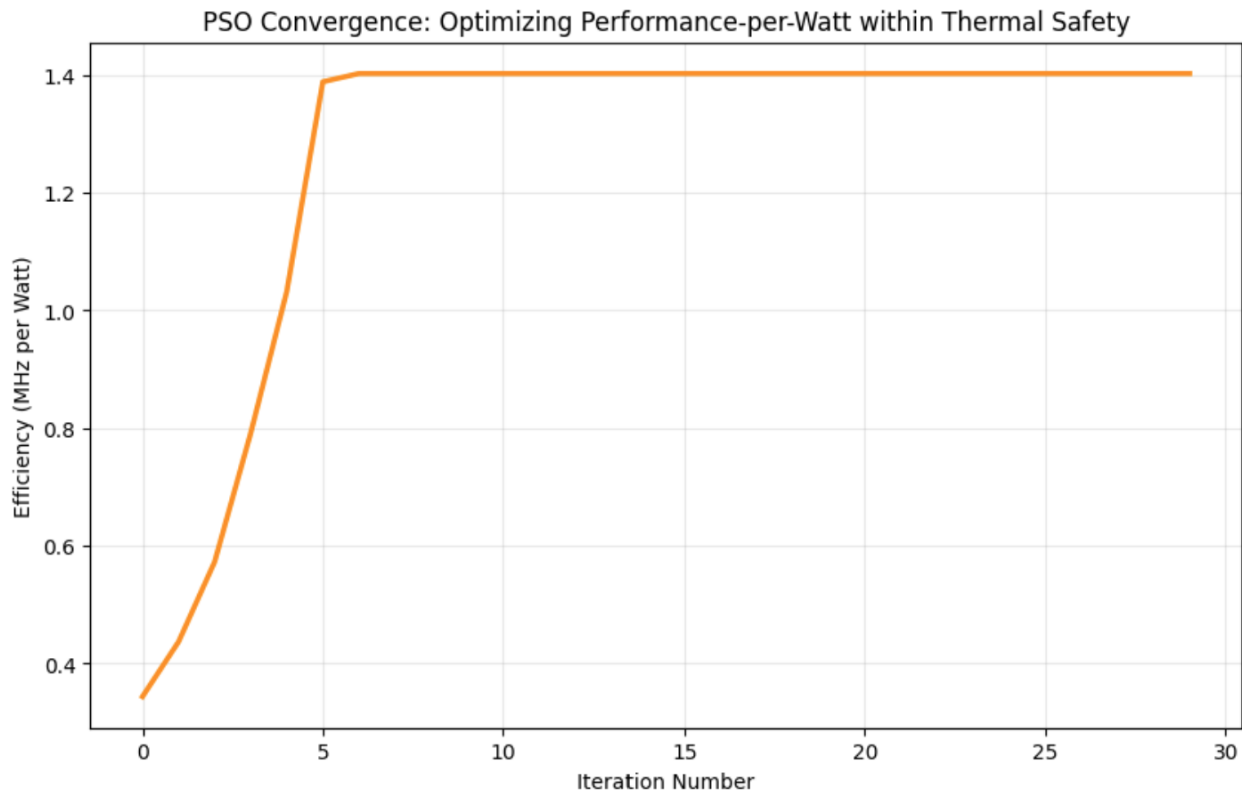


Figure 8: PSO Graph showing Iteration Number vs Efficiency

The optimal configuration achieved was a clock speed of 3.97MHz and a power consumption of 2.83W, resulting in a maximized efficiency of 1.4028 MHz/W. This result reveals a significant preference for energy-efficient "undervolting" strategies in sustained Bitcoin mining workloads. The PSO demonstrates that the highest performance-per-watt is found at the lower bounds of the GPU's operational range, specifically where power draw is minimized. This suggests that for high-performance computing data centers, aggressively pushing for maximum clock speeds often leads to diminishing returns due to disproportionate increases in power consumption and heat generation. Operating at this optimized state not only maximizes computational throughput per unit of energy but also minimizes thermal stress, which is a critical factor in preserving the long-term lifespan of GPU hardware under consistent, strenuous loads. While the PSO successfully maximized the performance-per-watt metric, the physical implication of the result is highly intuitive: operating at minimum power yields the highest efficiency, analogous to walking rather

than running a fixed distance to conserve energy. From an industrial standpoint, the algorithm's convergence on an extremely low clock speed is impractical for data centers. Running high-performance GPUs at fractional speeds results in severe capital underutilization and fails to meet compute throughput requirements. Nevertheless, the primary contribution of this investigation is the methodology itself. The constrained optimization framework successfully demonstrates how parameter customization can yield actionable, highly relevant operational bounds, making the approach adaptable as dimensionality and complexity increase.

The PSO algorithm demonstrated exceptionally fast convergence, reaching a stable global optimum within approximately 5 to 7 iterations. This efficiency is partially attributed to the well-defined, bounded search space which constrained the 20 particles to physical hardware limits derived from the data. Furthermore, the chosen hyperparameters ($w = 1.1$, $c1 = 1.5$, $c2 = 1.5$) were selected from established "good ranges" in literature to balance exploration and exploitation, though they were not formally tuned for this specific application. The implementation of a "heavy penalty" landscape also played a critical role. By assigning a fitness of $-1e9$ to any configuration violating the 5-degree thermal safety margin, the swarm was immediately repelled from unsafe regions, effectively pruning the search space. However, the speed of convergence also suggests a relatively simple fitness landscape, likely caused by the low variance in the target temperature (3.04°C standard deviation) and a limited number of unique values in the core clock data, which may have oversimplified the optimization task.

Conclusion

This investigation proved the feasibility of developing a thermal management system for GPU's when used in Bitcoin mining and other high-performance situations. Taking the 24, 957 rows of GPU performance metrics, the pipeline implemented standardization, PCA, regression, and Particle Swarm Optimization, all from mathematical first principles, except for the polynomial feature generation which was implemented using the sklearn Python library.

The 90% cumulative sum variance threshold was accounted for by four principal components, with the major component encapsulating the linked changes in fan response, clock speed, and power. The ridge polynomial regression of degree-2 and with a μ of 1 was the best regression model, with a MAE of 1.175°C and a R^2 value of 0.718. This was notably better than the next

best linear regression models which had a MAE of 1.586 °C and R^2 value of 0.607 and shows the value in including second-order feature interactions in this thermal management system.

Using PSO, the optimal configuration achieved was a clock speed of 3.97MHz and a power consumption of 2.83W, resulting in a maximized efficiency of 1.4028 MHz/W while ensuring to maintain a 5 °C safety margin to remain below the throttling threshold.

Within the optimization section of the project, future iterations of this study should prioritize formal hyperparameter tuning (e.g., using a grid search similar to the regression stage) for the PSO variables to ensure the convergence speed is not masking a sub-optimal local peak. A significant limitation encountered was the high swarm density relative to the low dimensionality of the dataset. Since many features were highly correlated, the effective search space was smaller than the eight dimensions initially proposed. Future work should involve incorporating more diverse, uncorrelated data points -such as varying memory timings, ambient temperatures, or different GPU architectures - to introduce higher dimensionality and complexity to the optimization problem. Additionally, investigating the "edge cases" where the target variable exhibits higher variance would provide a more robust test for the predictive model and the PSO's ability to maintain thermal safety under volatile workloads.

Collectively, the results of this investigation prove that machine learning and optimization techniques such as PSO can achieve applicable operational guidelines for GPU hardware.

References

- [1] A. Cano, “A survey on graphic processing unit computing for large-scale data mining,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 1, p. e1232, Nov. 2017, doi: <https://doi.org/10.1002/widm.1232>.
- [2] S. Seth, “GPU Usage in Cryptocurrency Mining,” *Investopedia*, Aug. 16, 2023. <https://www.investopedia.com/tech/gpu-cryptocurrency-mining/>
- [3] A. Payong, “Future Trends in GPU Technology | DigitalOcean,” *Digitalocean.com*, 2024. <https://www.digitalocean.com/community/conceptual-articles/future-trends-in-gpu-technology>
- [4] P. beker, “GPU Performance Metrics for Usage Scenarios,” *Kaggle.com*, 2025. <https://www.kaggle.com/datasets/peterbeker/gpu-performance-metrics-for-usage-scenarios> (accessed Apr. 05, 2026).
- [5] IBM, “What is principal component analysis (PCA)?,” *Ibm.com*, Dec. 08, 2023. <https://www.ibm.com/think/topics/principal-component-analysis>
- [6] D. Groth, S. Hartmann, S. Klie, and J. Selbig, “Principal components analysis,” *Methods in Molecular Biology (Clifton, N.J.)*, vol. 930, pp. 527–547, 2013, doi: https://doi.org/10.1007/978-1-62703-059-5_22.
- [7] D. Andrés, “Issue #91 - Principal Component Analysis (PCA),” *Substack.com*, Mar. 22, 2025. <https://mlpills.substack.com/p/issue-91-principal-component-analysis>
- [8] Y. Shi and R. Eberhart, “A modified particle swarm optimizer,” *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98TH8360)*, 2019, doi: <https://doi.org/10.1109/iccc.1998.699146>.
- [9] A. P. Engelbrecht, “Computational Intelligence,” Oct. 2007, doi: <https://doi.org/10.1002/9780470512517>.
- [10] I. T. Jolliffe and J. Cadima, “Principal component analysis: a review and recent developments,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, Apr. 2016, doi: <https://doi.org/10.1098/rsta.2015.0202>.
- [11] W. Clem. Karl, “Regularization in Image Restoration and Reconstruction,” *Elsevier eBooks*, pp. 183–V, Jan. 2005, doi: <https://doi.org/10.1016/b978-012119792-6/50075-9>.
- [12] Q. Bai, “(PDF) analysis of particle swarm optimization algorithm,” *Computer and Information Science*, https://www.researchgate.net/publication/42385717_Analysis_of_Particle_Swarm_Optimization_Algorithm (accessed Apr. 7, 2026).

- [13] G. Bhat, Suat Gumusoy, and U. Y. Ogras, “Analysis and Control of Power–Temperature Dynamics in Heterogeneous Multiprocessors,” *IEEE Transactions on Control Systems Technology*, vol. 29, no. 1, pp. 329–341, Feb. 2020, doi: <https://doi.org/10.1109/tcst.2020.2974421>.

Appendix A: Additional Figures

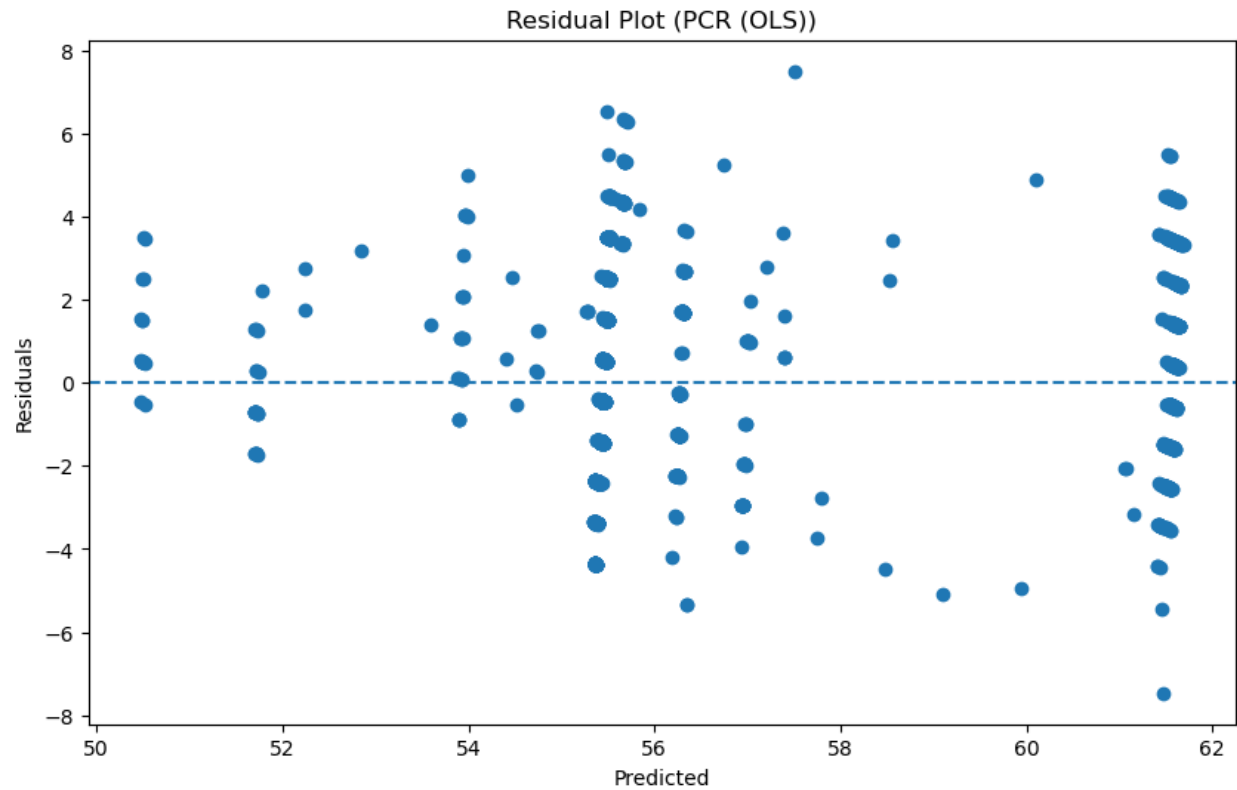


Figure A1: residual plot for PCR using OLS

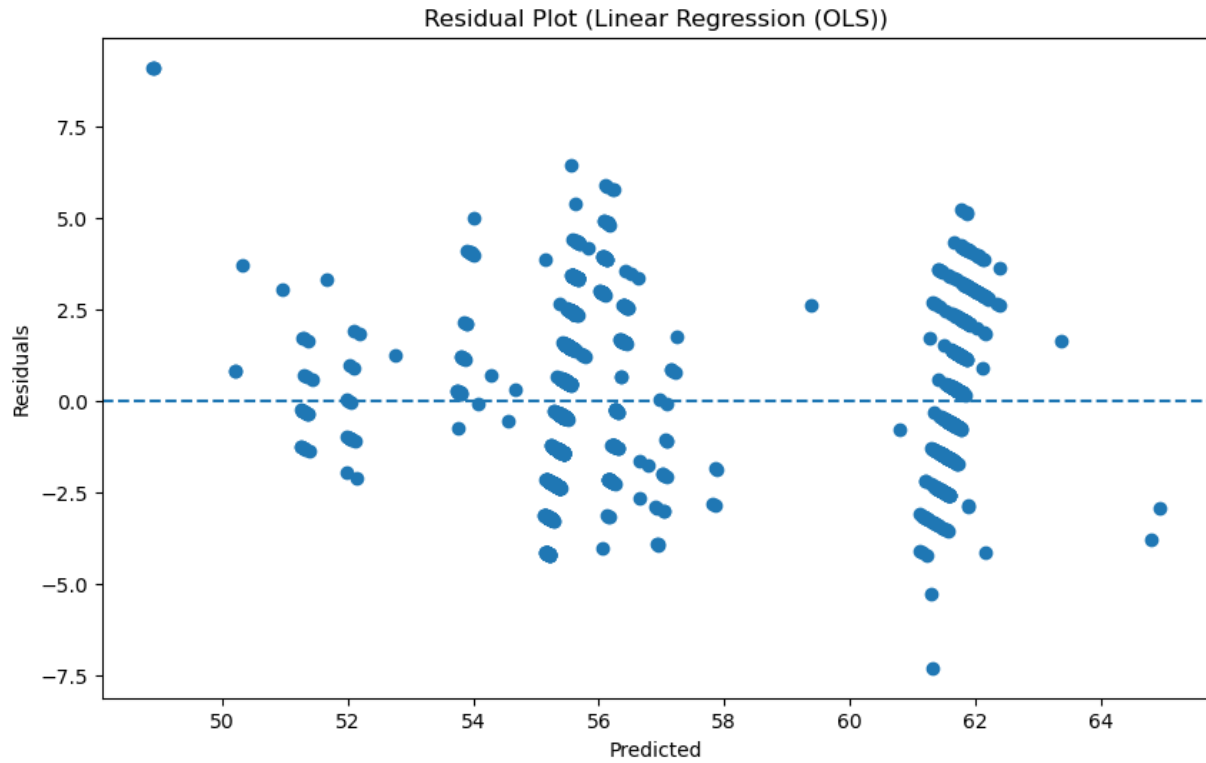


Figure A2: residual plot for Linear Regression using OLS



Figure A3: residual plot for Ridge Regression using raw features

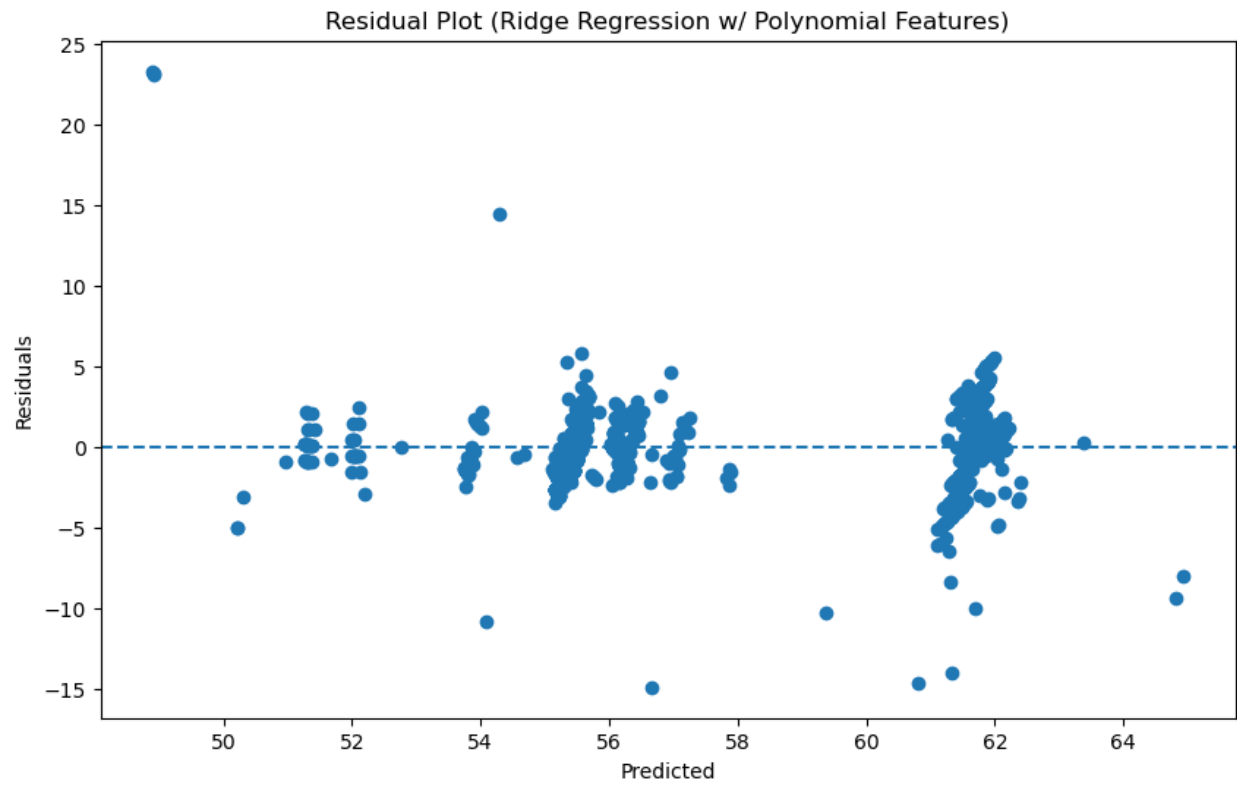


Figure A4: residual plot using Ridge Regression with polynomial features

Appendix B: Python Code

Part 1: PCA

```
In [189...  
import pandas as pd  
import numpy as np  
import seaborn as sns  
import matplotlib.pyplot as plt  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA
```

Step 1: Load Csv File and Drop Unused Columns

- Read csv file
- Drop columns for which the values are constant (provide no more info)

```
In [190...  
# read csv file  
df = pd.read_csv("data/BitCoin_Mining.csv")  
print(df.describe())  
  
# drop columns that are not used  
df = df.drop(columns=["timestamp", "mem_total", "latency_ms"])  
df = df.dropna()
```

	mem_used	mem_total	power_watts	gpu_util	temperature	\
count	24957.000000	24957.0	24957.000000	24957.000000	24957.000000	
mean	3.960139	4.0	30.493423	99.013704	56.431262	
std	0.192408	0.0	8.577073	9.703038	3.039341	
min	0.160000	4.0	2.830000	0.000000	50.000000	
25%	3.970000	4.0	26.940000	100.000000	54.000000	
50%	3.970000	4.0	27.120000	100.000000	56.000000	
75%	3.970000	4.0	27.470000	100.000000	58.000000	
max	3.970000	4.0	50.430000	100.000000	67.000000	

	fan_speed	core_clock	memory_clock	pcie_rx_MB	pcie_tx_MB	\
count	24957.000000	24957.000000	24957.000000	24957.000000	24957.000000	
mean	32765.687062	1772.709460	5954.746965	1.657050	0.02929	
std	207.415158	132.401724	500.983538	0.759343	0.16886	
min	0.000000	300.000000	405.000000	0.000000	0.00000	
25%	32767.000000	1785.000000	6000.000000	2.000000	0.00000	
50%	32767.000000	1785.000000	6000.000000	2.000000	0.00000	
75%	32767.000000	1785.000000	6000.000000	2.000000	0.00000	
max	32767.000000	1785.000000	6000.000000	3.000000	2.00000	

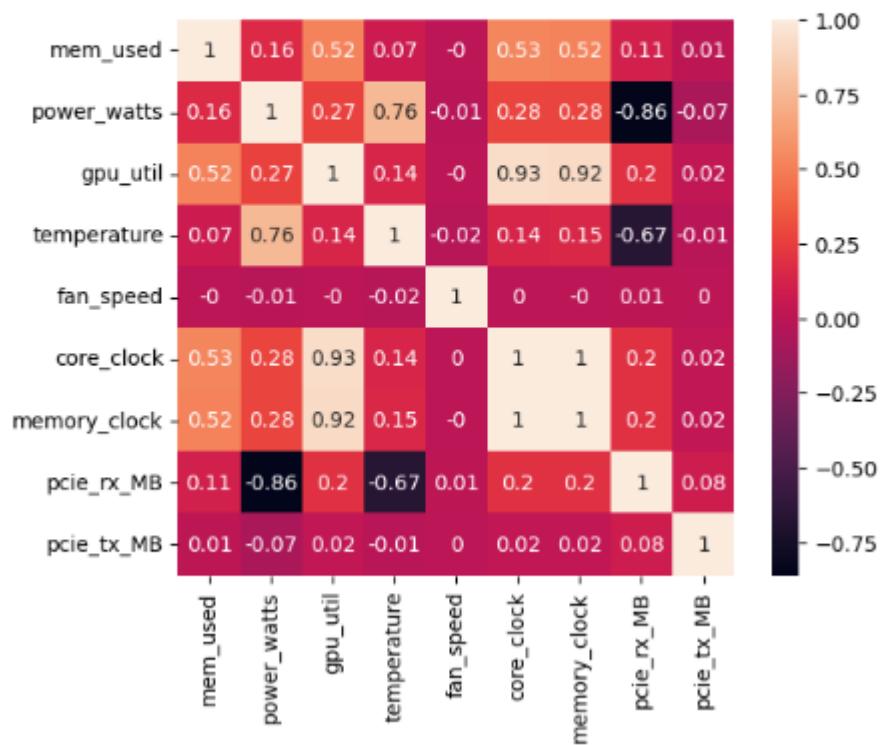
	latency_ms
count	24957.0
mean	0.0
std	0.0
min	0.0
25%	0.0
50%	0.0
75%	0.0
max	0.0

Step 2: Look at Correlation Matrix

- Calculate correlation coefficient between variables
- Create a heatmap to observe relationships

In [191]

```
co_mtx = round(df.corr(numeric_only=True), 2)
sns.heatmap(co_mtx, annot=True, square=True)
plt.show()
```



Step 3: Scaling Data for PCA

Scale each column based on:

$$x_i = \frac{x_i - \mu}{\sigma}$$

- x_i is the i 'th value in column x
- μ is the **mean** of the column (x)
- σ is the **standard deviation** of the column (x)

In [192_

```
# convert to df to numpy array
features = df.drop(columns=['temperature'])
X_data = features.values.astype(float)

# get mean of each column
mean_cols = X_data.mean(axis=0)

# get std
std_cols = X_data.std(axis=0)

scaled_X_data = (X_data - mean_cols) / std_cols

# check mean and std are 0 and 1
scaled_df = pd.DataFrame(scaled_X_data, columns=features.columns)
scaled_df.describe().round(3)
```

Out[192_

	mem_used	power_watts	gpu_util	fan_speed	core_clock	memory_clock	pcie_rx_MB	pcie_tx_MB
count	24957.000	24957.000	24957.000	24957.000	24957.000	24957.000	24957.000	24957.000
mean	0.000	-0.000	-0.000	-0.000	0.000	0.000	-0.000	-0.000
std	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
min	-19.751	-3.225	-10.205	-157.975	-11.123	-11.078	-2.182	-0.173
25%	0.051	-0.414	0.102	0.006	0.093	0.090	0.452	-0.173
50%	0.051	-0.393	0.102	0.006	0.093	0.090	0.452	-0.173
75%	0.051	-0.353	0.102	0.006	0.093	0.090	0.452	-0.173
max	0.051	2.324	0.102	0.006	0.093	0.090	1.769	11.671

Step 4: Perform PCA

- Compute SVD
- Compute explained variance for each principal component, can be done using:

$$\text{variance}_i = \frac{\sigma_i^2}{n-1}$$

$$\text{explained variance} = \frac{\text{variance}_i}{\text{total variance}}$$

source: <https://molgorithm.medium.com/how-to-calculate-explained-variance-ratio-in-pca-using-svd-ab531b434e6>

- where σ_i is the singular value
- where n = number of rows in the X matrix
- Create cumulative variance plot to determine number of principle components
- Retain X principal components and use $\tilde{X} = TP^T$ to compute new matrix

perform SVD (Economy)

```
U, Sigma, Vt = np.linalg.svd(scaled_X_data, full_matrices=False)
```

compute explained variance

```
variances = (Sigma**2) / (scaled_X_data.shape[0] - 1)
explained_variances = variances/variances.sum()
```

compute cumulative variance

```
cumulative_variance = np.cumsum(explained_variances)
```

make a plot to determine number of principal components based on threshold

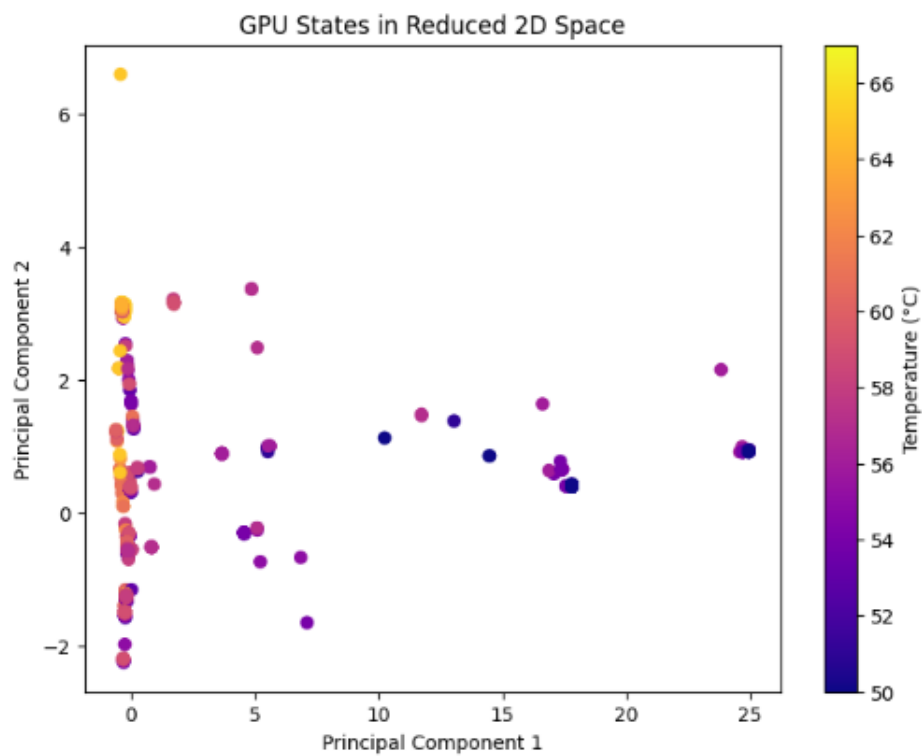
```
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(cumulative_variance) + 1), cumulative_variance, marker='o', linestyle='--')
plt.axhline(y=0.9, color='r', linestyle='-', label='90% Threshold')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Comparing Number of Principal Components to Their Cumulative Explained Variance ')
plt.legend()
plt.show()
```

In [194]

```
# num of principal components
k = 4
U_hat = U[:, :k]
Sigma_hat = Sigma[:k]
V_hat = Vt[:, :].T

# calculate T scores
T = U_hat * Sigma_hat
pca_result = T

# plot the first two principal components along with temperaure
plt.figure(figsize=(8,6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c=df['temperature'], cmap='plasma')
plt.colorbar(label='Temperature (°C)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('GPU States in Reduced 2D Space')
plt.show()
```



Part 2 : Regression

```
In [195... from sklearn.preprocessing import PolynomialFeatures
```

Step 1: Create Training and Test Data

- 80/20 split for training and test
- shuffle data to get random samples in each training/test dataset

```
In [196... X = X_data
# convert to numpy array
y = df['temperature'].values.astype(float)

# get number of vals in test/test
num_rows = X.shape[0]
num_vals_train = int(0.8 * num_rows)
num_vals_test = num_rows - num_vals_train

# random seed to reproduce
np.random.seed(20)

# get indices for each
shuffled_rows = np.random.permutation(num_rows)
train_idx = shuffled_rows[:num_vals_train]
test_idx = shuffled_rows[num_vals_train:]

X_train_raw = X[train_idx]
X_test_raw = X[test_idx]
# adding the bias column of ones
X_train = np.hstack([np.ones((X_train_raw.shape[0], 1)), X_train_raw])
X_test = np.hstack([np.ones((X_test_raw.shape[0], 1)), X_test_raw])

y_train = y[train_idx]
y_test = y[test_idx]

print(f"Dataset Size: {X.shape[0]}")
print(f"Train Size: {X_train.shape[0]}")
print(f"Test Size: {X_test.shape[0]}")
```

```
Dataset Size: 24957
Train Size: 19965
Test Size: 4992
```

Training/Test Data Using Principal Components

```
1 [197..
# Use PCA results from earlier
X_pcr = pca_result

# shuffle idxs
X_train_pcr_raw = X_pcr[train_idxs]
X_test_pcr_raw = X_pcr[test_idxs]

# Add bias column of ones
X_train_pcr = np.hstack([np.ones((X_train_pcr_raw.shape[0], 1)), X_train_pcr_raw])
X_test_pcr = np.hstack([np.ones((X_test_pcr_raw.shape[0], 1)), X_test_pcr_raw])
```

Performing Linear (OLS) and Ridge Regression using Raw Data

As defined in lecture Ridge-Regression is:

$$\mathbf{a}^* := [\Theta^T \Theta + \mu \mathbf{I}]^{-1} \Theta^T \mathbf{y} \quad \text{for some } \mu > 0$$

and Linear Regression using (ridge):

$$\mathbf{a}^* := [\Theta^T \Theta]^{-1} \Theta^T \mathbf{y}$$

We can perform these and fit the model to see how good they perform on test data

```
1 [198..
# function to perform ordinary Least squares (OLS)
def ols(theta, y):
    theta_T_theta = np.matmul(theta.T, theta)
    theta_T_y = np.matmul(theta.T, y)
    a_star = np.linalg.solve(theta_T_theta, theta_T_y)
    return a_star

# function to perform ridge regression
def ridge(theta, y, mu):
    theta_T_theta = np.matmul(theta.T, theta)
    theta_T_y = np.matmul(theta.T, y)
    a_star = np.linalg.solve(theta_T_theta + (mu * np.eye(theta_T_theta.shape[0])), theta_T_y)
    return a_star
```

Training data and creating predictions to compare to test data

PCR + OLS

```
[199..
a_star_pcr = ols(X_train_pcr, y_train)
y_pred_pcr = np.matmul(X_test_pcr, a_star_pcr)
# get MAE and R^2 value
mae_pcr = np.mean(np.abs(y_test - y_pred_pcr))
r_squared_pcr = 1 - np.sum((y_test - y_pred_pcr)**2) / np.sum((y_test - y_test.mean())**2)

# create plot to show comparison of actual vs predicted for OLS model
plt.figure(figsize=[10,6])
plt.figure(figsize=[10,6])
plt.scatter(y_test, y_pred_pcr, alpha=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', label='Perfect fit')
plt.xlabel('Actual Temperature (C)')
plt.ylabel('Predicted Temperature (C)')
plt.title(f'PCR (OLS) \n(MAE = {mae_pcr:.2f} & R^2 = {r_squared_pcr:.2f})')
plt.legend()

# residual plot
residuals = y_test - y_pred_pcr
plt.figure(figsize=[10,6])
plt.scatter(y_pred_pcr, residuals)
plt.axhline(0, linestyle='--')
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residual Plot (PCR (OLS))')
plt.show()
```

OLS

00_

```
a_star_ols = ols(X_train, y_train)
y_pred_ols = np.matmul(X_test, a_star_ols)
# get MAE and R^2 value
mae_ols = np.mean(np.abs(y_test - y_pred_ols))
r_squared_ols = 1 - np.sum((y_test - y_pred_ols)**2) / np.sum((y_test - y_test.mean())**2)

# create plot to show comparison of actual vs predicted for OLS model
plt.figure(figsize=[10,6])
plt.figure(figsize=[10,6])
plt.scatter(y_test, y_pred_ols, alpha=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', label='Perfect fit')
plt.xlabel('Actual Temperature (C)')
plt.ylabel('Predicted Temperature (C)')
plt.title(f'Linear Regression (OLS) \n(MAE = {mae_ols:.2f} & R^2 = {r_squared_ols:.2f})')
plt.legend()

#residual plot
residuals = y_test - y_pred_ols
plt.figure(figsize=[10,6])
plt.scatter(y_pred_ols, residuals)
plt.axhline(0, linestyle='--')
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residual Plot (Linear Regression (OLS))')
plt.show()
```

Ridge Regression

```
# perform some basic parameter tuning on penalty term mu
mu_values = [0.001, 0.05, 0.1, 1, 5, 10, 100, 1000]
res = []
for mu in mu_values:
    a_star_ridge = ridge(X_train, y_train, mu)
    y_pred_ridge = np.matmul(X_test, a_star_ridge)
    # get MAE and R^2 value
    mae_ridge = np.mean(np.abs(y_test - y_pred_ridge))
    r_squared_ridge = 1 - np.sum((y_test - y_pred_ridge)**2) / np.sum((y_test - y_test.mean())**2)
    res.append({"mu":mu, "mae":mae_ridge, "r2":r_squared_ridge})

# get mu corresponding to lowest MAE
best_mu = min(res, key=lambda x: x["mae"])["mu"]
print(best_mu)
print(res)

1.001
{'mu': 0.001, 'mae': np.float64(1.5858387402001892), 'r2': np.float64(0.6072768461481985)}, {'mu': 0.05, 'mae': np.float64(1.5
r2': np.float64(0.6062241022957392)}, {'mu': 5, 'mae': np.float64(1.5878580685511763), 'r2': np.float64(0.6053338006045581)},
': np.float64(1.6042096113705087), 'r2': np.float64(0.5994611657342837)}}
```

```
a_star_ridge = ridge(X_train, y_train, best_mu)
y_pred_ridge = np.matmul(X_test, a_star_ridge)
mae_ridge = np.mean(np.abs(y_test - y_pred_ridge))
r_squared_ridge = 1 - np.sum((y_test - y_pred_ridge)**2) / np.sum((y_test - y_test.mean())**2)
# create plot to show comparison of actual vs predicted for OLS model
plt.figure(figsize=[10,6])
plt.scatter(y_test, y_pred_ridge, alpha=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', label='Perfect fit')
plt.xlabel('Actual Temperature (C)')
plt.ylabel('Predicted Temperature (C)')
plt.title(f'Ridge Regression ( $\mu = \{best\_mu\}$ ) \n(MAE = {mae_ridge:.2f} & R^2 = {r_squared_ridge:.2f})')
plt.legend()

#residual plot
residuals = y_test - y_pred_ridge
plt.figure(figsize=[10,6])
plt.scatter(y_pred_ridge, residuals)
plt.axhline(0, linestyle='--')
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residual Plot (Ridge Regression)')
plt.show()
```

The ridge regression yields near identical results to the ols regression, so we can try using polynomial feature space to see if it yields better outcomes. We will use ridge because it did have a slightly better MAE.

```
10.. poly = PolynomialFeatures(degree=2, include_bias=True)
X_train_poly = poly.fit_transform(X_train_raw)
X_test_poly = poly.transform(X_test_raw)

# perform some basic parameter tuning on penalty term mu
mu_values = [0.001, 0.05, 0.1, 1, 5, 10, 100, 1000]
res_poly = []
for mu in mu_values:
    a_star_ridge_poly = ridge(X_train_poly, y_train, mu)
    y_pred_ridge = np.matmul(X_test_poly, a_star_ridge_poly)
    # get MAE and R^2 value
    mae_ridge_poly = np.mean(np.abs(y_test - y_pred_ridge))
    r_squared_ridge_poly = 1 - np.sum((y_test - y_pred_ridge)**2) / np.sum((y_test - y_test.mean())**2)
    res_poly.append({'mu':mu, 'mae':mae_ridge_poly, 'r2':r_squared_ridge_poly})

# get mu corresponding to lowest MAE
best_mu_poly = min(res_poly, key=lambda x: x["mae"])[ "mu" ]
print(best_mu_poly)
print(res_poly)

1
[{'mu': 0.001, 'mae': np.float64(1.1816283816267086), 'r2': np.float64(0.6135789411946146)}, {'mu': 0.05, 'mae': np.float64(1.1794763025967405), 'r2': np.float64(0.657936458446463)}, {'mu': 0.1, 'mae': np.float64(0.7177031445758177)}, {'mu': 5, 'mae': np.float64(1.178453179511963), 'r2': np.float64(0.7119615872727167)}, {'mu': 10, 'mae': np.float64(1.178775277705181), 'r2': np.float64(1.1779247549850034)}, {'mu': 100, 'mae': np.float64(0.7154914569151818)}, {'mu': 1000, 'mae': np.float64(0.7154914569151818)}]]
```

Use best μ to perform ridge regression (Poly Features)

```
11.. a_star_ridge_poly = ridge(X_train_poly, y_train, best_mu_poly)
y_pred_ridge_poly = np.matmul(X_test_poly, a_star_ridge_poly)
mae_ridge_poly = np.mean(np.abs(y_test - y_pred_ridge_poly))
r_squared_ridge_poly = 1 - np.sum((y_test - y_pred_ridge_poly)**2) / np.sum((y_test - y_test.mean())**2)
# create plot to show comparison of actual vs predicted for OLS model
plt.figure(figsize=[10,6])
plt.scatter(y_test, y_pred_ridge_poly, alpha=0.5)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         'r--', label='Perfect fit')
plt.xlabel('Actual Temperature (C)')
plt.ylabel('Predicted Temperature (C)')
plt.title(f'Ridge Regression Using Polynomial Features( $\mu = \{best\_mu\_poly\}$ ) \n(MAE = {mae_ridge_poly:.4f} & R^2 = {r_squared_ridge_poly:.4f})')
plt.legend()

#residual plot
residuals = y_test - y_pred_ridge_poly
plt.figure(figsize=[10,6])
plt.scatter(y_pred_ols, residuals)
plt.axhline(0, linestyle='--')
plt.xlabel('Predicted')
plt.ylabel('Residuals')
plt.title('Residual Plot (Ridge Regression w/ Polynomial Features)')
plt.show()
```

Part 3: Particle Swarm Optimization

2.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. PSO Parameters and Initialization
n_particles = 20
iterations = 30
w = 1.1 # Inertia weight
c1 = 1.5 # Personal best
c2 = 1.5 # Global best
history = []

CLOCK_IDX = 0
POWER_IDX = 1

# Define bounds from existing scaled/raw data
bounds_min = X_data.min(axis=0)
bounds_max = X_data.max(axis=0)

particles_pos = np.random.uniform(bounds_min, bounds_max, (n_particles, X_data.shape[1]))
particles_vel = np.zeros((n_particles, X_data.shape[1]))
p_best_pos = np.copy(particles_pos)
p_best_fitness = np.full(n_particles, -np.inf)
g_best_pos = np.zeros(X_data.shape[1])
g_best_fitness = -np.inf
```

3.

```
# 2. Optimization Loop
for i in range(iterations):
    for j in range(n_particles):
        # Transform particle position into Polynomial Features for the Ridge model
        poly_pos = poly.transform(particles_pos[j].reshape(1, -1))

        # Predict temperature using a_star_ridge_poly weights
        pred_temp = np.matmul(poly_pos, a_star_ridge_poly)

        current_clock = particles_pos[j, CLOCK_IDX]
        current_power = particles_pos[j, POWER_IDX]

        # Implementing the safety margin as 5 degrees under throttle point
        if pred_temp < (83 - 5):
            # Efficiency metric (Perf/Watt)
            # Avoid division by zero with a small epsilon
            fitness = current_clock / (current_power + 1e-7)
        else:
            # Heavy penalty for violating safety margin
            fitness = -1e9

        # Update Personal Best
        if fitness > p_best_fitness[j]:
            p_best_fitness[j] = fitness
            p_best_pos[j] = np.copy(particles_pos[j])

        # Update Global Best
        current_g_best_idx = np.argmax(p_best_fitness)
        if p_best_fitness[current_g_best_idx] > g_best_fitness:
            g_best_fitness = p_best_fitness[current_g_best_idx]
            g_best_pos = np.copy(p_best_pos[current_g_best_idx])

        # 3. Update Velocities and Positions
        r1, r2 = np.random.rand(), np.random.rand()
        particles_vel = (w * particles_vel +
                        c1 * r1 * (p_best_pos - particles_pos) +
                        c2 * r2 * (g_best_pos - particles_pos))

        # Clip positions to ensure the swarm stays within physical hardware limits
        particles_pos = np.clip(particles_pos + particles_vel, bounds_min, bounds_max)

    history.append(g_best_fitness)
```

```
# 4. Results
print(f"Optimal Efficiency Achieved: {g_best_fitness:.4f} MHz/W")
print(f"Optimal Configuration: Clock={g_best_pos[CLOCK_IDX]:.2f}MHz, Power={g_best_pos[POWER_IDX]:.2f}W")

# Visualization
plt.figure(figsize=(10, 6))
plt.plot(history, color='darkorange', linewidth=2.5)
plt.xlabel('Iteration Number')
plt.ylabel('Efficiency (MHz per Watt)')
plt.title('PSO Convergence: Optimizing Performance-per-Watt within Thermal Safety')
plt.grid(True, alpha=0.3)
plt.show()
```